Assignment

C# for Java Programmers

GenAI Level 1 — Limited AI Use ⚠️

This assignment is classified as **Level 1 — Limited AI Use**. Please follow the **Level 1 GenAI requirements** described in the *Generative AI Use in Assessments* policy posted at the top of the Assignments section in Moodle.

Scenario — Smart Parking Permit Management System

A municipality currently manages parking permits using spreadsheets and manual records. As the number of residents and vehicles grows, staff need a small application that can store permit information, validate permit data, calculate permit fees, and display permit details.

You have been asked to develop a **C# console application using .NET** called the **Smart Parking Permit Management System**.

Because you already have experience with Java, the goal is not to relearn basic programming. Instead, this assignment focuses on applying familiar object-oriented concepts using **C# syntax and C#-specific features**, particularly properties, interfaces, structs, and enums. This reflects the central goal of the provided material: translating existing Java programming knowledge into C# and recognizing the important differences between the two languages.

---

1. Learning Objectives

After completing this assignment, you should be able to:

- create and run a C# console application;
- use common C# data types, operators, conditions, and methods;
- create C# classes and objects;
- use constructors;
- use C# properties instead of traditional Java-style getter/setter methods;
- implement validation through property accessors;
- define and implement a C# interface;
- define and use an enum;
- define and use a struct;

- recognize important syntactic differences between Java and C#;
- compile, run, test, and explain your C# program.

---

2. Create the C# Project

Using Visual Studio, create a new **C# Console App (.NET)** project named:

`ParkingPermitSystem`

Before implementing the complete application, run the initial project and verify that it compiles and executes correctly.

Your application should use **`Console.WriteLine()`** to display information to the user. As discussed in the course material, this is the C# equivalent commonly encountered when translating Java code that uses **`System.out.println()`**.

---

3. Part A — Create the ParkingPermit Class

Create a class named:

`ParkingPermit`

The class should represent a parking permit issued to a resident.

Include appropriate data for each permit, such as:

- **`Permit ID`**
- **`Owner Name`**
- **`Vehicle Plate`**
- **`Vehicle Type`**
- **`Monthly Fee`**
- **`Number of Months`**
- **`Permit Status`**

Choose appropriate C# data types for these values.

For example, your implementation should make meaningful use of types such as:

- **`int`**
- **`string`**
- **`double`**
- **`bool`**

Note
Many C# types are familiar to Java programmers but that some names differ—for example, Java's boolean corresponds to C#'s bool, and Java's String is normally written as string in C#.

---

4. Part B — Constructor and Objects

Create an appropriate constructor for the **`ParkingPermit`** class.

The constructor should receive the information required to initialize a new parking permit.

Then create **at least five different parking permit objects** containing realistic sample data.

Your objects should represent different owners, vehicles, permit durations, fees, and statuses.

---

5. Part C — Use C# Properties

At least some of the data in your **`ParkingPermit`** class must be accessed through **C# properties**.

For example, instead of following the traditional Java approach:

`getMonthlyFee()`

`setMonthlyFee()`

use an appropriate C# **`property`**.

At least **one property must contain validation logic**.

For example, your program could prevent:

- a negative monthly fee;
- an invalid number of months; or
- another invalid value appropriate to your design.

Practical Requirement

Demonstrate the validation by attempting to assign both:

1. a valid value; and
2. an invalid value.

Your output should make it clear whether the invalid value was accepted or rejected.

---

6. Part D — Create and Use an Enum

Create an **`enum`** named:

`PermitStatus`

Use it to represent a fixed set of possible permit states.

For example:

`Active`

`Expired`

`Suspended`

You may use different meaningful values if appropriate.

Do **not** represent these statuses using arbitrary integers such as:

```
int status = 2;
```

Use the **`enum`** throughout your application.

---

7. Part E — Create and Use a Struct

Create a struct named:

`PermitPeriod`

Use it to represent information related to the permit period.

For example, it could contain:

- **`Start Year`**
- **`Start Month`**
- **`Duration in Months`**

Create at least one **`PermitPeriod`** value and associate or display it with a parking permit.

Your implementation should demonstrate that you understand the basic purpose and syntax of a C# struct, as introduced in the course material.

---

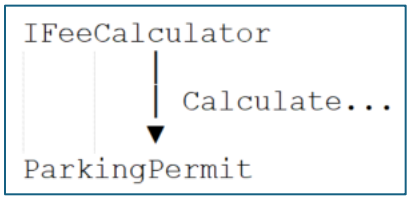
8. Part F — Create and Implement an Interface

Create an interface named:

`IFeeCalculator`

The interface should declare a method responsible for calculating a permit's total cost.

For example, conceptually:



Have your **`ParkingPermit`** class implement this interface.

Remember the C# syntax discussed in the course material:

```
class SomeClass : ISomeInterface
```

rather than Java's:

```
class SomeClass implements SomeInterface
```

Interfaces serve the same general purpose in both languages—a contract that implementing classes must follow—but the implementation syntax differs.

---

9. Part G — Calculate Permit Costs

Use your interface method to calculate the total cost of each permit.

For this assignment, use the following basic calculation:

Total Permit Cost = Monthly Fee × Number of Months

For example:

Monthly Fee = $45
Duration = 6 months

Total = 45 × 6
= $270
Display the calculated cost for each permit.

---

10. Part H — Use Operators and Conditional Logic

Your application must demonstrate meaningful use of C# operators covered in the course material.

Include examples of:

- arithmetic operators;
- comparison operators;
- logical operators; and
- assignment operators.

For example, determine whether a permit satisfies multiple conditions before displaying a message:

Permit is active
AND
Duration is valid
↓
Permit is currently valid

The condition must be meaningful in the context of the parking permit system rather than included only to demonstrate syntax.

---

11. Part I — Display Permit Information

Create a method that displays the information for a parking permit.

Your final application should display meaningful output similar to:

=== SMART PARKING PERMIT SYSTEM ===

Permit ID: 1001
Owner: Alex Martin
Vehicle Plate: ABC123
Vehicle Type: Car
Status: Active
Monthly Fee: $45.00
Duration: 6 months
Total Cost: $270.00

Display information for **all five permits**.

---

12. Part J — Java-to-C# Comparison

Java vs. C# Observations

Identify **at least four differences or similarities** that you encountered while completing the assignment.

Your comparison should be based on your actual implementation.

For example, you could discuss:

- `System.out.println()` versus `Console.WriteLine();`
- `boolean` versus `bool`;
- `String` versus `string`;
- Java `getter/setter` methods versus C# `properties`;
- `implements` versus `:` for interfaces;
- constructors;
- classes;
- operators;
- `enums` or `structs`.

NOTE
Do not simply copy the comparison from the course notes. Explain what you observed while developing your own application.

---

13. Testing Requirements

Compile and run the completed application.

Your testing must demonstrate at least:

- successful creation of five parking permit objects;
- correct use of C# data types;
- constructor execution;
- property access;
- property validation with both valid and invalid data;
- enum usage;
- struct usage;
- interface implementation;
- correct permit-cost calculations;

- conditional logic;
- successful final execution.

Test more than one case. A program that merely compiles is **not sufficient evidence that the application works correctly**.

---

14. Report and Screenshot Requirements

Submit a **short but complete report documenting your practical work**.

The report must include a **comprehensive set of screenshots demonstrating that you completed, executed, and tested the practical requirements yourself**.

Your screenshots should clearly demonstrate:

1. creation and structure of the Visual Studio/.NET project;
2. the ParkingPermit class;
3. constructor implementation;
4. properties and property validation;
5. the PermitStatus enum;
6. the PermitPeriod struct;
7. the IFeeCalculator interface;
8. implementation of the interface by your class;
9. creation of at least five permit objects;
10. permit-cost calculations;
11. use of operators and conditional logic;
12. testing of valid and invalid property values;
13. successful compilation and execution;
14. final console output.

Screenshots must be **clear, readable, logically organized, and accompanied by a short caption or explanation**.

Important: Showing only the final source code and final console output is not sufficient. Your report must provide evidence that the practical requirements were implemented and tested.

Also include your **Java vs. C# Observations** and the **required GenAI Disclosure Statement** according to the Level 1 policy posted on Moodle.

---

15. Submission Requirements

Submit the following through **Moodle**:

1. Your complete Visual Studio C# project, including all source code.
2. A report that includes a comprehensive set of screenshots demonstrating the implementation, execution, and testing of the practical requirements.

All files should be clearly named and organized.

You must respect the due date specified on Moodle.

---

16. Mandatory Presentation to the Professor

In addition to submitting your work on Moodle, you must **present and demonstrate your completed assignment to the professor to receive a grade**.

During the presentation, you may be asked to:

- run your application;
- explain your ParkingPermit class;
- explain a property and its get/set behavior;
- explain your validation logic;
- explain the purpose of your interface;
- explain your enum and struct;
- explain a Java/C# difference;
- explain a calculation or conditional statement;
- make a small modification to your program and run it again.

Important Grading Requirement

Submission on Moodle alone is not sufficient. The assignment must also be presented to the professor to receive a grade. An assignment that is submitted but not presented will not be graded.

---

17. Assessment Rubric

Criterion	Weight	Expectations
C# Application & OOP	30%	Correct project structure, classes, objects, constructors, data types, methods, and working application
C# Properties & Validation	20%	Correct use of properties and meaningful validation using get/set
Interface, Enum & Struct	20%	Correct and meaningful implementation and use of the three C# constructs
Program Logic & Testing	15%	Correct calculations, operators, conditions, execution, and testing of valid/invalid cases
Report, Java/C# Comparison & Screenshots	15%	Clear report, meaningful Java/C# observations, and comprehensive evidence of practical work
Total	**100%**	

Mandatory GenAI Disclosure Requirement

For this **Level 1 assessment**, the required **GenAI Disclosure Statement must be included in the report**, whether GenAI was used or not.

Failure to include the required GenAI Disclosure Statement will result in a grade of zero (0) for the assignment.